Project Task: PayLedger (Digital Wallet App)
Oct 6, 2026 · Sahin Mondal
Assigned To: Amit Kumar Verma

Project

Build **PayLedger**, a simple digital wallet app. Users can add money, send money to other users, and see their history. An admin can approve withdrawals.

You have **2 days**. You can use AI tools. But you must understand every line of code you submit.

Tech to use

- Next.js (frontend
- Node.js + Express + TypeScript (backend)
- PostgreSQL with Drizzle ORM
- Docker Compose to run everything

What to build

User features

1. Register and login.
2. Wallet page that shows the balance.
3. Add money. Build a fake payment gateway yourself. After payment, it calls your webhook and the money is added.
4. Send money to another user.
5. Transaction history with pagination.
6. Withdraw request. The money is held until an admin approves or rejects it.

Admin features

1. See all withdrawal requests and approve or reject them.
2. Reverse a transfer (money goes back to the sender).

Money rules

- Store money as whole paise (integers). Never use decimals or floats.
- Transfer fee is 1%, minimum ₹1, maximum ₹20. The sender pays it.
- Balance can never go below zero.
- Maximum ₹25,000 per transfer and ₹50,000 per day per user.
- Save every money movement as ledger entries. The entries of one transaction must add up to zero. Never edit or delete ledger entries.

This must work correctly

1. **No double payment.** If the user clicks twice or the request is retried, money moves only once. Use an Idempotency-Key header.
2. **Parallel requests.** A wallet has ₹1,000. Fifty transfers of ₹100 (₹101 with fee) arrive at the same time. Only 9 should succeed, and the balance should end at ₹91.
3. **All or nothing.** A transfer is saved in one database transaction. No half-done money.
4. **Webhook safety.** Check the signature. If the same webhook comes 5 times, add money only once.
5. **Privacy.** A user can see only their own wallet and transactions.

What to submit

- A private GitHub repo with commits made step by step
- README with simple steps to run it (docker compose up)
- Tests for the five points above (npm test, using a real PostgreSQL database)
- DECISIONS.md: a short note on why you built it this way, and where AI made mistakes and how you fixed them
- A 5 minute video demo

After you submit

We will have a call. You will share your screen, explain your code, and make one small change live.